

SESSION 5

Context Engineering I — Retrieval

The field shifted from wording prompts to engineering what fills the window.

Mercredi 23 septembre · 8h00-9h30 · 1h30

Prompt & Context Engineering — Stefan Duprey

5

TODAY

Session 5 — plan & goals

You will be able to

- Keep secrets out of Git (.gitignore, .env)
- Reframe prompting as context engineering
- Ground answers with retrieval + citation
- Handle context rot / lost-in-the-middle

RUN OF SHOW

- | | |
|------|--------------------------------|
| 0:00 | Git — secrets & .gitignore |
| 0:15 | Context engineering: retrieval |
| 0:40 | Lab — grounded, measured RAG |

Keep secrets out of Git

- API keys live in .env — never in your code or notebooks
- Add .env to .gitignore BEFORE your first commit
- A key pushed to GitHub is compromised — rotate it
- Load secrets at runtime from the environment

the safe pattern

```
# .gitignore
.env
*.key
```

```
# load at runtime, not in code
load_dotenv()
key = os.environ["OPENAI_API_KEY"]
```

From prompt to context engineering

Here's the mental shift that defines the modern job — from polishing sentences to deciding what fills the window.

- **The prompt is only part of the window**

Retrieved documents, examples, conversation history and tool results all share the context — and all compete for attention.

- **The real skill is curation**

Deciding what goes in, and in what order, matters more than polishing the exact sentence. This is context engineering.

- **It's the field's current framing**

As models got better at following instructions, the bottleneck moved from wording to context assembly. That's where the work is now.



SHIFT

Stop polishing sentences; start curating context.
What's in the window matters more than the exact phrasing around it.

Retrieval as context assembly

RAG isn't magic; it's four plain steps that end in 'assemble the right context, then ask'.

- 1 Load & split**
Chunk documents into passages of a few hundred tokens. Chunk boundaries strongly affect what you can retrieve.
- 2 Embed & store**
Turn each chunk into a vector and index it for similarity search. Local embeddings keep this offline.
- 3 Retrieve**
Embed the question, fetch the top-k most similar chunks. This is nearest-neighbour search, not keyword matching.
- 4 Assemble**
Place the retrieved chunks plus the question into the prompt — carefully, because order and volume matter.

Embeddings & vector search

The engine under retrieval is simpler than it sounds — a way to find text that means the same thing, not just matching words.

- **Embeddings map meaning to vectors**

Text with similar meaning lands at nearby points. That's what lets us find relevant passages without exact keyword matches.

- **Retrieval is nearest-neighbour search**

Embed the query, find the closest chunk vectors. Simple, fast, and the backbone of RAG.

- **Chunking is a real lever**

Chunk size and overlap change what can be retrieved. Too big buries the answer; too small loses context.



WHY

Grounding the model in retrieved text is the single most practical cure for hallucination — give it the facts instead of hoping it remembers them.

Grounding, citation, refusal

Three small instructions turn a confident guesser into a source-backed answerer — here's the workhorse prompt.

- **Answer only from the context**

Instruct the model to use the retrieved chunks and nothing else. This is what makes the answer traceable.

- **Require citations**

Ask for [1], [2] references to the chunks. Citations let users verify and make the model stay grounded.

- **Permit 'I don't know'**

Explicitly allow refusal when the answer isn't in the context. This clause is the strongest anti-hallucination lever you have.

grounded QA prompt

Answer using ONLY the context.
Cite sources as [1], [2].
If the answer isn't there, say
"I do not know."

Context: {chunks}

Question: {q}

Context rot & lost-in-the-middle

More context feels safer, but it can quietly make answers worse — here's the catch nobody warns you about.

- **More context is not always better**

Dumping everything in dilutes the signal and can lower answer quality. Retrieval is about relevance, not volume.

- **The middle gets ignored**

Models attend most to the start and end of a long context; facts buried in the middle are often missed.

- **Irrelevant chunks actively distract**

A wrong-but-plausible retrieved passage can pull the answer off course. Precision beats recall here.



MEASURE IT

Retrieval quality is an eval problem too — score grounding and refusal with your S3 harness instead of eyeballing a few answers.

RAG failure modes & fixes

When your RAG bot gives a bad answer it's usually one of these four — and each has a concrete fix.



Bad chunks

Answers split across chunk boundaries. Fix by tuning size/overlap and splitting on structure, not fixed length.



Wrong retrieval

The right passage never gets fetched. Try query rewriting or hybrid keyword + vector search.



Ignored context

The model answers from priors, not the chunks. Restate the grounding rule and place context near the question.



No citations

Unverifiable answers. Number the chunks and require inline [n] references in the output.

Lab — Grounded, measured RAG

LOCAL · OLLAMA

`prompt-engineering-course/05_context_engineering`

TASKS

1. Keep API keys in `.env`, never commit them (the Git touch-in)
2. Index a small document set with local embeddings
3. Write a grounded prompt that cites sources and can refuse
4. Score grounding and refusal on a small eval set



DELIVERABLE

An offline RAG notebook that cites its sources, refuses when the answer is absent, and reports a grounding score.

Recap — Session 5



Prompting is really context engineering



Grounding + citation + refusal fight hallucination



More context can hurt — mind context rot



Score retrieval quality; don't eyeball it

NEXT SESSION

Context engineering II

Tools and memory are context too. Tool schemas as prompts, memory as curated context, prompting inside agents.